Intelligent Systems (2802ICT) Assignment 1

Roman Green (s5369389)

Task 1 – Solving a Maze

1. Problem Formulation

Task 1 involves reading a maze from a file and searching for a path from a starting position to a goal position. The maze is modelled as an occupancy grid with N rows and M columns. The problem may be defined formally as an agentic search problem as follows:

State space	The grid-world position of the agent formulated as a pair of non-negative integers. I.e., $S = (r, c) \in \mathbb{N}_0 \times \mathbb{N}_0 : 0 \leq r < \mathrm{N}; 0 \leq c < \mathrm{M}$.
Initial state	The grid coordinates of the "A" character in the maze file. I.e., $s_0 = (5, 0)$ in maze1.txt.
Goal state	The grid coordinates of the "B" character in the maze file. I.e., $s_{goal} = (0, 5)$ in maze1.txt.
Actions	For any s, the agent may move up, $down$, $left$ and $right$ where the target cell is not occupied. I.e., $Actions = \{up, down, left, right\}$.
Transition model	For each action, the agent state may increment or decrement the row or column for the corresponding direction. I.e., $Result\big((r,c), up\big) = (r-1, c)$ $\quad Result\big((r,c), down\big) = (r+1, c)$, $Result\big((r,c), left\big) = (r, c-1)$, $\quad Result\big((r,c), right\big) = (r, c+1)$
Path-cost function	As move cost is 1, path cost is the number of moves taken to reach the current state.

2. Program Design and Data Structures

2.2. State Representation

Agent state is represented as a Python tuple of two integers, row and column respectively.

2.3. Data Structures

`Node`	A Python dataclass storing the node state, a reference to the parent node, the action taken to reach the node and the node depth.
`FrontierADT`	An abstract datatype defining necessary method node management on the frontier.
`StackFrontier`	A `FrontierADT` implementation leveraging a last-in-last-out stack for use in the depth-limited depth-first search method.
`PriorityQueueFrontier`	A `FrontierADT` implementation leveraging a priority-heap for use in the A* search method. On construction it accepts a method cost_fn to compute node cost.

2.4. Search Procedure Implementation

Three search procedures were implemented: depth-limited depth-first search (DLS), iterative depth-first search (IDS) leveraging DLS and A* search (A*).

2.4.1. Depth-limited depth-first search

Implemented as the `Maze.dfs_solve` method, DLS is an uninformed pre-order tree-like search leveraging the `StackFrontier` class. On failure to find the goal node the method will raise `DepthLimitReachedError` and `NoSolutionError` where a depth cutoff does and does not occur respectively. The method is complete provided the goal depth does not exceed the `max_depth` argument and is not optimal.

DLS is implemented as follows. A starting node is defined with the starting state, no parent, no action and a depth of 0. A `StackFrontier` is defined and prepopulated with the starting node. A cutoff occurred tracking variable is defined. While the frontier is not empty, a node is popped from the frontier and a goal check is made. Then for each neighbour, if neighbour depth exceeds max depth, the neighbour is skipped and the cutoff occurred tracking variable is triggered. If the neighbour already exists in the path to the current node (cycle tracking) it is skipped, otherwise it is added to the frontier.

2.4.2. Iterative depth-first search

Implemented as the `Maze.ids_solve` method, IDS leverages DLS to perform an uninformed but complete and optimal graph-like search. Starting at depth 1, IDS will repeatedly call DLS with increasing depths until either a solution is found, or some max search depth is reached.

2.4.3. A* search

A* is an informed graph-like search algorithm utilising a configurable heuristic and the `PriorityQueueFrontier` class. Two separate A* methods were implemented as `Maze.astar_solve` and `Maze.astar_solve_consisent` to handle consistent and non-consistent heuristics. The consistent variant (`Maze.astar_solve_consisent`) is complete and optimal provided the heuristic is consistent. The general variant (`Maze.astar_solve`) is complete and optimal provided the heuristic is admissible. The reason for the two separate implementations is that the consistent variations utilises both an early goal check and an expanded set to prevent no re-expansion.

The general A* is implemented as follows. A starting node is defined with the starting state, no parent, no action and a depth of 0. A `PriorityQueueFrontier` is defined and prepopulated with the starting node, a heuristic function, and a tiebreaker function. For all heuristic configurations of A*, negative node depth was used as the tiebreaker function to prefer exploration of nodes closer to the goal. A reached dictionary is defined to keep track of reached nodes and their associated path costs. A solution variable is defined to keep track of the best solution found thus far. While the frontier is not empty, a node is popped from the frontier. If the node state equals the goal state and the node cost is less than the current stored solution cost, the node is assigned to the solution variable. If the solution variable cost is less than the lowest cost present on the frontier, the algorithm returns. Otherwise, for each neighbour of the current node, if the neighbour state is not in the reached dictionary or if the neighbour cost is less than a node with the same state but higher cost it is added to the frontier and the reached dictionary is updated.

2.5. Heuristic

Three heuristics were implemented: the Manhattan heuristic, the Euclidian heuristic and a convolutional-neural-network (CNN) based heuristic.

2.5.1. Manhattan Heuristic

The Manhattan heuristic was chosen as it provides a fast, consistent, admissible heuristic to inform A*. The Manhattan heuristic may be defined as follows:

$$h(n) = |x_n - x_{goal}| + |y_n - y_{goal}|$$

For a heuristic $h(n)$ to be consistent, for all nodes n and n', $h(n)$ must not exceed the path cost from n to n' and the heuristic cost $h(n')$. That is, $\forall\, n, \forall\, n'$ successor of n: $h(n) \leq c(n, n') + h(n')$.

To show consistency, consider moving from n to a neighbour n', the Manhattan heuristic guarantees $|h(n) - h(n')| \leq 1$. This can be rewritten as $h(n) \leq 1 + h(n') = c(n, n') + h(n')$, therefore satisfying the requirements for consistency.

While consistency implies admissibility, I will comment on the admissibility of the Manhattan heuristic. Assume the maze world is 4-connected with no obstacles. The shortest distance from the starting node to the goal point would be the difference in horizontal and vertical components as is in the Manhattan heuristic. When reintroducing obstacles to the world, the optimal path length will either be the same or increased. Therefore, in our 4-connected world with obstacles, the Manhattan heuristic will not exceed the optimal path length, meeting the requirements for admissibility ($\forall\, n$: $h(n) \leq h^*(n)$ where $h^*(n)$ is the optimal path length from n to the goal).

2.5.2. CNN Heuristic

Implemented as a "special feature/extension" for this assignment. In 2019, Yuka Ariki and Takuya Narihira showed that a small encoder-decoder neural network could be used to generate heuristic maps to inform A* search algorithms on a small grid-based world. (Ariki & Narihira, 2019) Therefore, as an additional heuristic for Task 1, I attempted to reproduce their results. The Python libraries Flax and Jax from Google were used to implement and train the model respectively. Relevant files are shown in Table 1.

The model accepts a 224x224x3 tensor where channel one is a binary obstacle map (0 for empty space, 1 for walls), channel two is a Euclidian transform of the obstacle map and, channel three is a euclidian distance map from the goal point (Figure 1, A). No input normalisation was performed. The output is a 224x224x1 tensor representing the heuristic value of each cell (Figure 1, B Prediction).

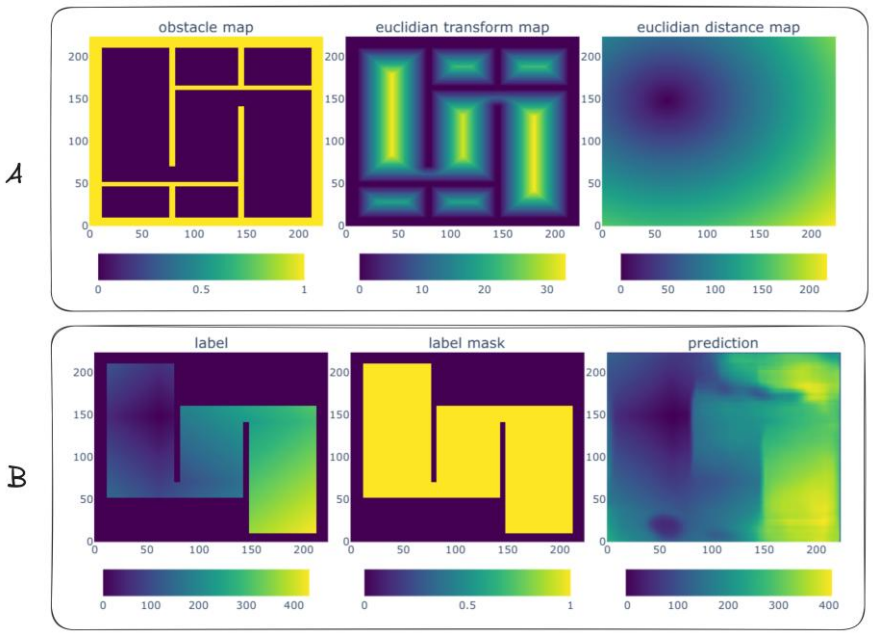


Figure 1 Sample model inputs and outputs. Sample input features shown on A. Sample label, label mask and model prediction shown on B.

The model was trained on the mazes subset of the motion planning dataset (Bhardwaj, 2017/2025) for 2200 epochs. The loss is the mean squared error evaluated only on elements covered by the label mask; elements where the mask is zero do not contribute to the loss.

The encoder-decoder architecture closely mirrors that of Ariki and Narihira. In short: three encoder stages (3→16→32→64) each using a strided 3×3 conv downsample followed by two dilated 3×3 convs, all with BatchNorm and leaky ReLU; and three symmetric decoder stages using a 4×4 transposed conv upsample followed by two 3×3 convs with BatchNorm and leaky ReLU; a final 1×1 conv produces the single output channel.

The CNN heuristic cannot be considered admissible or consistent.

3. Experimental Results

Experimental results tables are provided in Appendix A “Experimental Results on Provided Mazes” and Appendix B “Experimental Results on CNN Heuristic Compatible Mazes.”

Appendix A features Iterative Depth-First Search and consistent A* with the Manhattan heuristic. A* with the CNN heuristic is omitted due to the maze dimensions being incompatible with the model.

Appendix B features A* with the Manhattan heuristic and CNN heuristic. Iterative Depth-First Search is omitted due to the long running time on mazes of 224x224 pixels.

The experimental results in Appendix A shows that both IDS and A* can identify optimal solutions to the given maze problems. Unsurprisingly IDS expands far more nodes compared to A* as it contains no mechanism to avoid node re-expansion and is uninformed.

The experimental results in Appendix B shows that the CNN heuristic is able to produce workable heuristic maps.

The CNN-guided A* tends to expand far more nodes than the Manhattan-heuristic variant. This can be improved by utilising a weighted A* search, or alternatively by training the model for a longer period of time. The original authors trained their models for 10,000 epoch, nearly 5 times the epoch count of the model used in this assignment.

The CNN-guided A* also runs noticeably faster than the Manhattan-heuristic version. This may be accounted by the GPU execution and XLA optimisations performed by JAX and the simple heuristic map lookup following inference for the CNN while the Manhattan heuristic suffers a pure Python overhead. Therefore, runtimes reported for CNN and the Manhattan heuristic cannot be reasonably compared due to underlying implementation differences.

4. Conclusion

Iterative depth-first search and A* algorithms were successfully implemented for the purpose of solving simple 4-connected mazes. CNN-guided A* offers no advantage over simpler heuristics such as the Manhattan heuristic or a Euclidian heuristic, however may offer an advantage in settings where simple, symbolic heuristics are not known.

Task 2 – Solving a Cross Word

1. Experimental Results

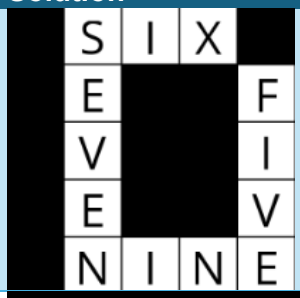
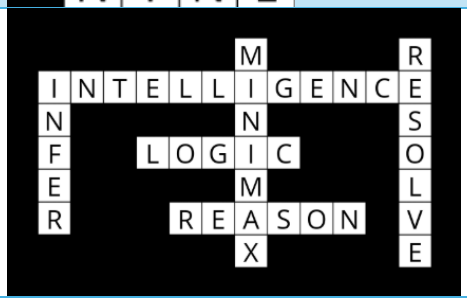
A backtracking crossword solver was successfully implemented to solve the 3 provided crosswords. The solver utilised Minimum Remaining Values (MRV) and Least Constraining Value (LCV) heuristic, while enforcing arc consistency using AC-3. The results are summarized below in Table 1.

Experiment 1 (55.8 μs) used source files `structure0.txt and words0.txt`. Due to the small word list of 10 words, after the initial node consistency filtering the domain for each variable was constricted to between 3 or 4 words. Furthermore, following arc consistency using AC-3 the domain for each variable was constricted to one word for three of the variables and two words for the final variable. Therefore, there was very little “search” for the backtracking algorithm to perform, and the solution was found almost instantly.

Experiment 2 (120 μs) used source files `structure1.txt and words1.txt`. Despite a more complex structure with more variables and constraints the performance remained extremely fast. As the word list only contained 51 words the search space was quite small. Following node consistency, each variable had at most 10 words in its domain. Following AC-3 pruning, each variable had at most 2 words in its domain. Therefore, similarly to experiment 1, there was very little “search” for the backtracking algorithm to perform, and the solution was found almost instantly.

Experiment 3 (99,400 μs) used source files `structure2.txt and words2.txt`. While the puzzle structure was of comparable complexity to the second experiment, the word list was significantly larger containing 3000 words. Consequently, initial node and arc consistency checks were far less effective. Following node consistency, each variable had close to 500 words in their domain. Following AC-3 pruning, each variable had its domain reduce by 10 to 100 words. Consequently, as the backtracking search was forced to explore a vast search tree with large domains for each variable, a slowdown of ~800x occurred compared to Experiment 2.

Table 1 Sample Crossword Solutions.

Crossword	Time Taken (μs)	Solution
structure0.txt words0.txt	55.8	
structure1.txt words1.txt	120	

structure2.txt words2.txt	99,400	

2. Special Features/Extensions

The ability to handle partially solved crosswords was implemented by modifying the crossword text loader method and introducing additional unary constraint checking in the `enforce_node_consistency` function. Sample outputs are shown in Table 2.

Table 2 Sample Crossword Solutions with additional Unary Constraints.

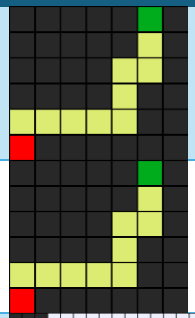
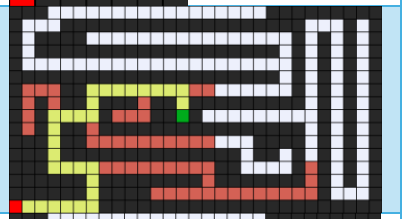
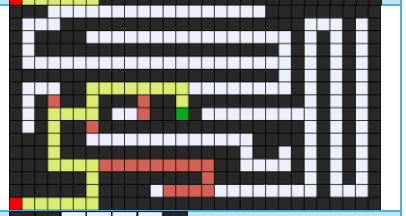
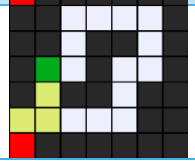
Crossword	Time Taken (μs)	Solution
######_ ____##_ _##rat_ _##_##_ _##_##_ #___##_ words2.txt	18,300	
######_ ____##_ _##k___ _##_##_ _##_##_ #___##_ words2.txt	917	
######_ d___##_ _##b___ _##_##_ _##_##_ #___##_ words2.txt	2,070	

References

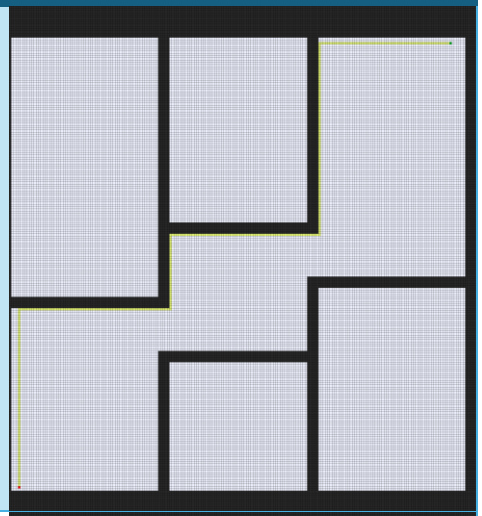
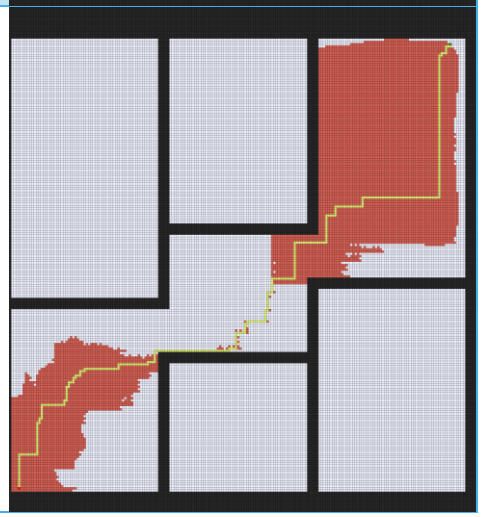
Ariki, Y., & Narihira, T. (2019). *Fully Convolutional Search Heuristic Learning for Rapid Path Planners* (No. arXiv:1908.03343). arXiv. https://doi.org/10.48550/arXiv.1908.03343

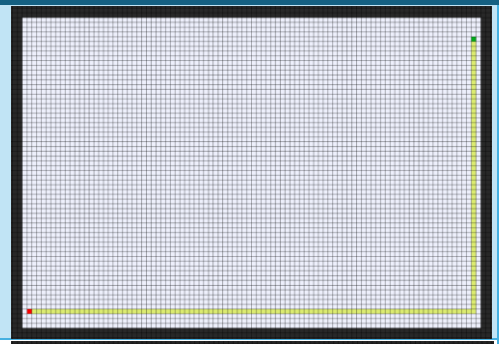
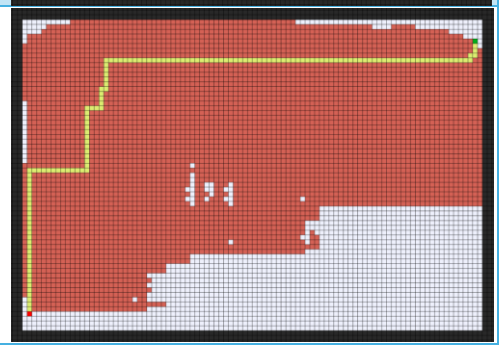
Bhardwaj, M. (2025). *Mohakbhardwaj/motion_planning_datasets* [Computer software]. https://github.com/mohakbhardwaj/motion_planning_datasets (Original work published 2017)

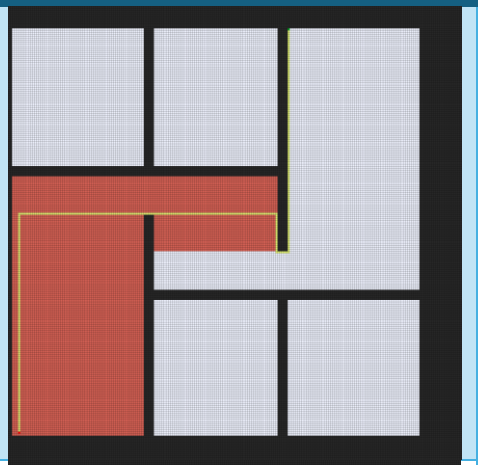
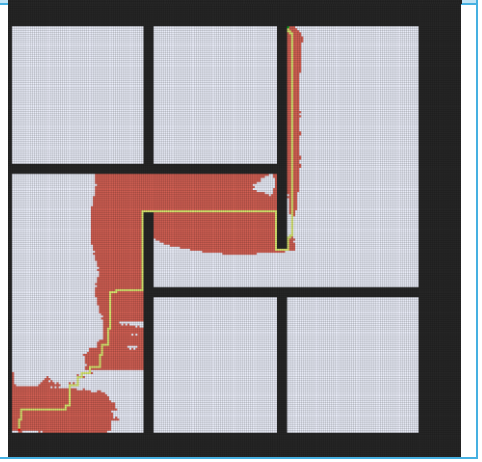
Appendix A. Experimental Results on Provided Mazes

Example	Algorithm	Heuristic	Path Cost	Is Optimal	States Explored	Time Taken (ms)	Visualisation
norm_1.txt	IDS	N/A	10	Yes	65	0.138	
norm_1.txt	A*	Manhattan	10	Yes	10	1.770	
norm_2.txt	IDS	N/A	30	Yes	909	2.080	
norm_2.txt	A*	Manhattan	30	Yes	48	17.400	
norm_3.txt	IDS	N/A	4	Yes	17	0.035	
norm_3.txt	A*	Manhattan	4	Yes	4	1.760	

Appendix B. Experimental Results on CNN Heuristic Compatible Mazes

Example	Algorithm	Heuristic	Path Cost	Is Optimal	States Explored	Time Taken (s)	Visualisation
cnn_0.txt	A*	Manhattan	388	Yes	388	1.990	
cnn_0.txt	A*	CNN	388	No	13656	0.025	

Example	Algorithm	Heuristic	Path Cost	Is Optimal	States Explored	Time Taken (s)	Visualisation
cnn_3.txt	A*	Manhattan	150	Yes	150	2.130	
cnn_3.txt	A*	CNN	152	No	153	0.011	

Example	Algorithm	Heuristic	Path Cost	Is Optimal	States Explored	Time Taken (s)	Visualisation
cnn_4.txt	A*	Manhattan	370	Yes	19536	2.234	
cnn_4.txt	A*	CNN	374	No	12809	0.013